



**UNIVERSIDAD COMPLUTENSE
MADRID**

Actividad de Machine learning

Autor: Gerardo Fructuoso Vidal-Aragón

Profesores/as: Inmaculada Gutiérrez, Daniel Gómez, Javier Castro

Indice

1.	Introducción	2
2.	Apartado 1: Análisis y depuración de datos.	3
a.	Tratamiento de datos missing	6
b.	Estandarización de datos.....	7
c.	Convertir variables categóricas a numéricas	7
3.	Apartado 2: Creación de modelo de arbol de decisión.....	8
4.	Apartado 3: Random forest y XGBOOST	12
a.	Random forest	12
b.	XGBOOST	16
5.	Apartado 4: Conclusión final	20

1. Introducción

En esta asignatura vamos a dar algunos de los distintos modelos de machine learning que hay junto con sus métricas para medir si el modelo está bien construido o no.

Entre estos tipos de modelos, principalmente vamos a ver:

- Modelo de clasificación
- Modelo de regresión
- Modelo de máquinas de vector soporte
- Modelos de árboles de decisión
- Creación de estos modelos con distintas técnicas de ensemble

Entre las distintas métricas que vamos a ver, destacan:

- Curva ROC
- Matriz de confusión
- MAE (Error absoluto medio)
- MSE (Error cuadrático medio)
- Otros muchos

2. Apartado 1: Análisis y depuración de datos.

En este apartado lo primero que hacemos es revisar el dataset que tenemos y estudiarlo un poco para ver que puede contener.

Por lo tanto, lo primero que se tiene que hacer es guardar los datos del csv en un dataframe.

Para analizarla, vemos primero la frecuencia que tiene la variable objetivo sobre nuestros registros y es lo siguiente:

```
La frecuencia de cada clase de la variable objetivo es:  
Y  
0    0.5  
1    0.5  
dtype: object
```

Como podemos ver es un 50% de 0 y 50% de 1.

Como los nombres de las variables son V1, V2, ... Y no se entienden correctamente, renombramos para que se entiendan mejor.

Ahora vemos los estadísticos de nuestro conjunto de datos, cantidad de filas y los tipos de las variables.

Cantidad de filas:

```
[9200 rows x 21 columns]  
Filas: 9200
```

Tipos:

```
Tipos: estado_clie      int64  
duracion_cuen          int64  
cod_area_tel           float64  
tel_clie               int64  
plan_inter             int64  
plan_buzon             int64  
count_buzon            int64  
mins_tot_diurnas       float64  
count_diurnas          int64  
coste_tot_diurnas      float64  
mins_tot_vespertinas   float64  
count_vespertinas      int64  
coste_tot_vespertinas  float64  
mins_tot_noc           float64  
count_noc              float64  
coste_tot_noc          float64  
mins_tot_inter         float64  
count_inter            int64  
coste_tot_inter        float64  
count_svc              int64  
Y                      int64  
dtype: object
```

Estadísticos:

Estadísticos:		count	mean	std	min	25%	50%	75%	max
estado_clie	9200.0	26.232391	14.697631	0.0	14.0000	26.00	39.0000	50.00	
duracion_cuen	9200.0	102.230326	40.065555	1.0	75.0000	102.00	128.0000	243.00	
cod_area_tel	9125.0	437.141370	42.320277	408.0	415.0000	415.00	510.0000	510.00	
tel_clie	9200.0	2512.690978	1438.955632	0.0	1253.0000	2536.50	3751.2500	4999.00	
plan_inter	9200.0	0.175326	0.380266	0.0	0.0000	0.00	0.0000	1.00	
plan_buzon	9200.0	0.217826	0.412791	0.0	0.0000	0.00	0.0000	1.00	
count_buzon	9200.0	6.517826	12.872573	0.0	0.0000	0.00	0.0000	52.00	
mins_tot_diurnas	9200.0	191.584424	61.523538	0.0	148.1000	189.30	236.7000	351.50	
count_diurnas	9200.0	100.207826	20.247057	0.0	87.0000	100.00	114.0000	160.00	
coste_tot_diurnas	9174.0	32.591252	10.458436	0.0	25.1825	32.20	40.2600	59.76	
mins_tot_vespertinas	9200.0	204.571446	51.337843	22.3	169.7750	206.20	239.6250	363.70	
count_vespertinas	9200.0	100.264783	19.319472	12.0	88.0000	100.00	113.0000	170.00	
coste_tot_vespertinas	9200.0	17.388750	4.363599	1.9	14.4275	17.53	20.3700	30.91	
mins_tot_noc	9128.0	202.685221	50.403908	0.0	169.2750	202.10	237.4000	395.00	
count_noc	9140.0	99.674836	20.108410	0.0	86.0000	100.00	114.0000	175.00	
coste_tot_noc	9200.0	9.125050	2.270690	0.0	7.6200	9.10	10.6825	17.77	
mins_tot_inter	9200.0	10.420228	2.791518	0.0	8.6000	10.40	12.3000	20.00	
count_inter	9200.0	4.402065	2.582138	0.0	3.0000	4.00	6.0000	20.00	
coste_tot_inter	9156.0	2.812696	0.754248	0.0	2.3200	2.81	3.3200	5.40	
count_svc	9200.0	1.878152	1.590014	0.0	1.0000	1.00	3.0000	9.00	
Y	9200.0	0.500000	0.500027	0.0	0.0000	0.50	1.0000	1.00	

Con esto a simple vista podríamos decir:

- No existen variables categóricas (todas son int o float aunque algunas sean binarias)
- Tenemos distintas variables binarias como plan_inter y plan_buzon
- Tenemos algunas variables que pueden parecer clases aunque no sean binarias (como podría ser cod_area_tel y estado_clie)
- Por las descripciones de la variable, podría parecer que el número del cliente fuese un dato único, pero como estamos viendo aquí, el máximo es 4999 mientras que hay 9200 registros.
- También vemos que hay datos faltantes

Viendo esto, primero revisamos bien la variable número de cliente, para ver si es un identificador único o no. Para ello, vemos si hay más de un registro con el mismo número:

```
Name: proportion, dtype: float64
tel_clie
4562    18
3965    15
1569    15
2696    15
2456    14
..
882      2
2269     2
434      2
3908     2
1376     2
Name: count, Length: 1021, dtype: int64
```

Como se puede ver en esta imagen vemos que, por ejemplo, el número 4562, aparece 18 veces dentro de los registros. Por lo tanto, no vamos a tomar que esta variable sea un dato único de los registros y no vamos a eliminarla.

Por otro lado, revisamos los datos faltantes de nuestro conjunto de datos:

```
Name: count, Length: 1921, dtype: int64
estado_clie          0
duracion_cuen        0
cod_area_tel        75
tel_clie             0
plan_inter           0
plan_buzon           0
count_buzon          0
mins_tot_diurnas     0
count_diurnas        0
coste_tot_diurnas    26
mins_tot_vespertinas 0
count_vespertinas    0
coste_tot_vespertinas 0
mins_tot_noc         72
count_noc            60
coste_tot_noc         0
mins_tot_inter       0
count_inter          0
coste_tot_inter      44
count_svc            0
Y                    0
```

Podemos ver que hay 5 variables que tienen datos faltantes.

Una vez que hemos terminado de analizar los datos que tenemos, procedemos a depurar estos datos para poder crear bien nuestro modelo.

Para ello, tenemos que seguir los siguientes pasos

3. Tratamiento de datos missing

Como no son muchos los valores faltantes, lo que he hecho ha sido agregar estos valores mediante imputación con la mediana de cada una de estas variables

```
columns_missing = ['cod_area_tel', 'coste_tot_diurnas', 'mins_tot_noc', 'count_noc', 'coste_tot_inter']

for x in columns_missing:
    df[x] = ImputacionCuant(var=df[x], tipo='mediana')

# comprobamos que ya no hay datos faltantes
print(df.isna().sum()) # ya no hya
```

Comprobamos que se ha realizado bien la imputación:

```
dtype: int64
estado_clie          0
duracion_cuen        0
cod_area_tel         0
tel_clie             0
plan_inter           0
plan_buzon           0
count_buzon          0
mins_tot_diurnas     0
count_diurnas        0
coste_tot_diurnas    0
mins_tot_vespertinas 0
count_vespertinas    0
coste_tot_vespertinas 0
mins_tot_noc         0
count_noc            0
coste_tot_noc        0
mins_tot_inter       0
count_inter          0
coste_tot_inter      0
count_svc            0
Y                   0
dtype: int64
```

Con esto vemos que ya no hay datos faltantes.

4. Estandarización de datos

Una vez que no tenemos datos faltantes, tendríamos que estandarizar estos datos. Para ello se utiliza la función de `StandardScaler` dentro de la librería `sklearn`.

Lo único, que solo estandarizaríamos las variables continuas numéricas. Por ello, he descartado tanto las binarias como las que parecían estar divididas en clases para estandarizarlas. SE ha realizado de la siguiente manera:

```
numericas_continuas =
['duracion_cuen','count_buzon','mins_tot_diurnas','count_diurnas','coste_tot_diurnas',
'mins_tot_vespertinas', 'count_vespertinas', 'coste_tot_vespertinas','mins_tot_noc','count_noc',
'coste_tot_noc', 'mins_tot_inter', 'count_inter','coste_tot_inter', 'count_svc']

scaler = StandardScaler()

for x in numericas_continuas:
    df[x] = scaler.fit_transform(df[[x]])

print(df.head()) # datos ya estandarizados
```

Con esto, ya tendríamos los datos estandarizados.

5. Convertir variables categóricas a numéricas

Como en este caso, en estos datos no tenemos variables categóricas, este paso no sería necesario realizarlo.

Con estos pasos ya habríamos realizado un análisis de los datos y los habríamos depurado y preparado para comenzar con la creación de los modelos.

6. Apartado 2: Creación de modelo de árbol de decisión

EN este apartado se nos pide crear un árbol de decisión que pueda clasificar y predecir la variable objetivo según los datos de entrada. Para ello, se han realizado los siguientes pasos (los dos primeros nos sirven para todos los modelos ya):

1. Separar la variable objetivo de las variables input

```
x = df.drop(['Y'],axis=1)
y = df['Y']
```

2. Dividir los datos en train y test. Para este apartado se ha dejado un 20% de los datos en test:

```
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=seed) #
dejamos un 20% para el conjunto test
```

3. Creación del árbol de decisión (lo he creado de clasificación en este caso) dejando como mínimo de muestras requeridas para dividir un nodo interno de 30. DE criterio he usado Gini:

```
arbol = DecisionTreeClassifier(min_samples_split=30, criterion='gini',random_state=seed)
```

4. Por último, entrenamos el modelo con las variables de entrenamiento

```
arbol.fit(X_train,y_train)
```

Una vez que tenemos creado el árbol de decisión vemos algunas de las métricas de clasificación para ver si el modelo es bueno:

Accuracy train: 0.9679347826086957

Accuracy test: 0.9434782608695652
Precisión train: 0.9587546567323044
Precisión test: 0.9255765199161425
Recall/sensibilidad train: 0.9780130293159609
Recall/sensibilidad test: 0.9639737991266376
F1_score train: 0.9682880945982263
F1_score test: 0.9443850267379679

Según vemos estas métricas, vemos lo siguiente:

- Accuracy: Este modelo ha acertado una proporción de predicciones totales muy alta.
- Precisión: El modelo predijo un 95,8% bien las clases de la variable objetivo con lo que es un alto porcentaje
- Recall: Vemos que el modelo es bastante sensible
- F1-Score: Tiene bastante buen score (al final esta es una mezcla entre precisión y recall).

Con esto estamos viendo que el modelo es muy bueno, tan bueno que igual esta sobreentrenado ya que no le hemos incluido ninguna profundidad como máxima y que el test da un poco más bajo las métricas conforme a lo que nos da el train. Para ver otro modelo que pueda ser mejor, aplicamos la función gridsearch y le incluimos varios parámetros.

Por primera vez se aplicaron los siguientes parámetros:

```
params = {  
'max_depth': [2, 3, 5, 10, 20],  
'min_samples_split': [5, 10, 20, 50, 100],  
'criterion': ["gini", "entropy"]  
}
```

Al ver que los modelos que nos devolvían eran muy buenos debido a su gran profundidad del árbol, opté por cambiar los parámetros reduciendo la profundidad del árbol para que sea más sencillo pero que siga valiendo los aciertos que nos da. Por ello, incluí los siguientes parámetros:

```
params2 = {  
'max_depth': [2, 3],  
'min_samples_split': [3,4,5,6,7,10, 20],  
'criterion': ["gini", "entropy"]  
}
```

Como resultado de búsqueda, nos quedamos con el siguiente modelo con los siguientes parámetros:

- Criterio: Gini
- Min_samples_split: 7
- Max_depth: 3

Con estos parámetros, nos han salido las siguientes métricas:

Se tiene un accuracy para train de: 0.8379076086956522

Se tiene un accuracy para test de: 0.8260869565217391

Precisión train: 0.8623799825429154

Precisión test: 0.8441108545034642

Recall/sensibilidad train: 0.8045602605863192

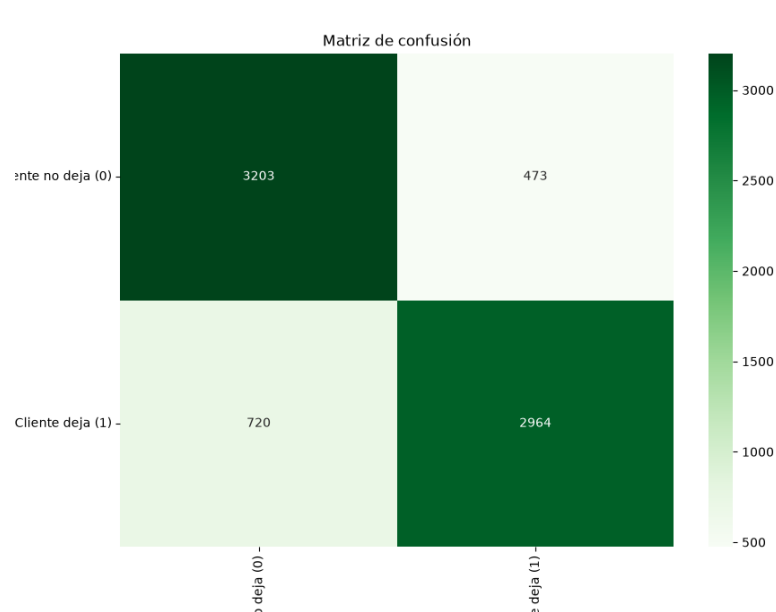
Recall/sensibilidad test: 0.7980349344978166

F1_score train: 0.8324673500912794

F1_score test: 0.8204264870931538

Como se puede observar, son un poco peores estas métricas (como un 10% peor) pero no está el árbol sobre entrenado ya que tiene muy poca profundidad.

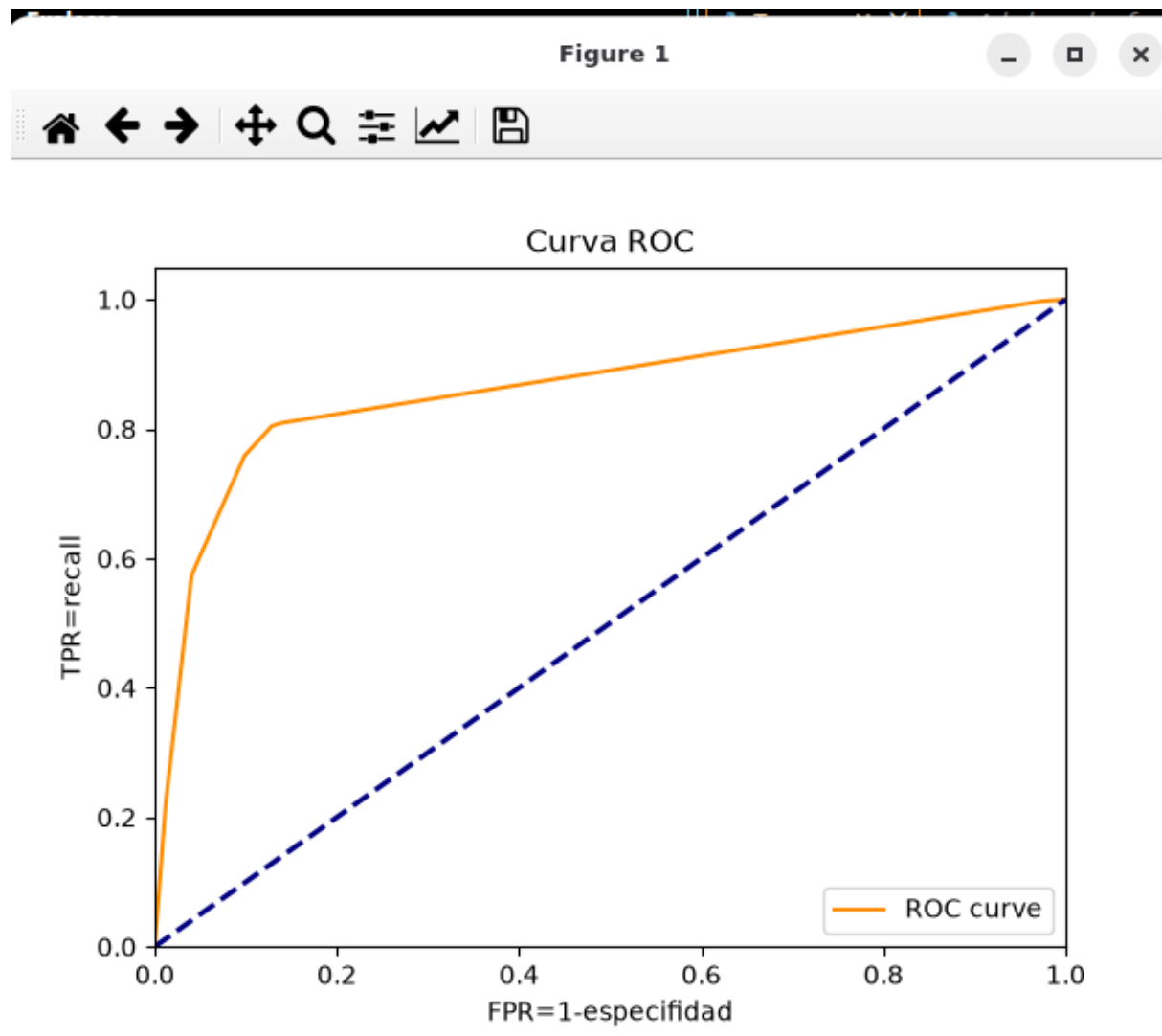
La matriz de confusión es la siguiente:



Con esto podemos observar que hay algunas clasificaciones que no hace muy bien, pero la mayoría sí que están bien clasificadas. Como vemos en esta matriz de confusión hay

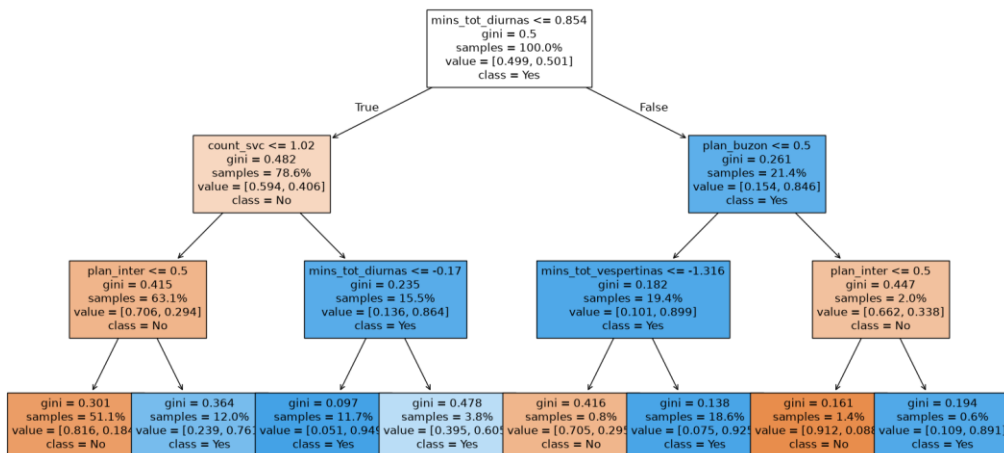
720 clientes que nos lo clasifica como que el cliente no nos deja cuando realmente no está dejando mientras que hay 473 que dice que nos deja cuando no nos está dejando.

La curva ROC que ha salido es la siguiente:



Como podemos ver en esta curva, parece que sigue un poco sobreentrenado, pero no tanto como estaba antes ya que antes solo se veía una línea arriba.

Por último, mostramos como quedaría el árbol gráficamente:



Como podemos ver en la imagen, igual le podríamos quitar una profundidad ya que a partir de la cuarta caja de la última fila, no encontramos casi ningún registro. Pero como en la parte de la izquierda de la última fila podemos ver que hay muchos, podríamos dejarlo así.

Por lo tanto, nos quedaríamos este modelo como árbol de decisión

7. Apartado 3: Random forest y XGBOOST

8. Random forest

Primero comenzamos creando el árbol de clasificación con random forest. Para ello, como hemos visto en el apartado anterior que si ponemos mucha profundidad se sobreentrena, vamos a ponerle un máximo de profundidad de 3. Con esto, lo entrenamos:

```
arbol_rf = RandomForestClassifier(max_depth = 3, criterion='gini', random_state=seed)
arbol_rf.fit(X_train, y_train)
```

Sacamos las métricas para ver si está bien entrenado este modelo:

Se tiene un accuracy para train de: 0.8793478260869565

Se tiene un accuracy para test de: 0.8777173913043478

Precisión train: 0.9021864211737629

Precisión test: 0.9022118742724098

Asignatura: Machine learning

Recall/sensibilidad train: 0.8512486427795874

Recall/sensibilidad test: 0.8460698689956332

F1_score train: 0.8759776536312849

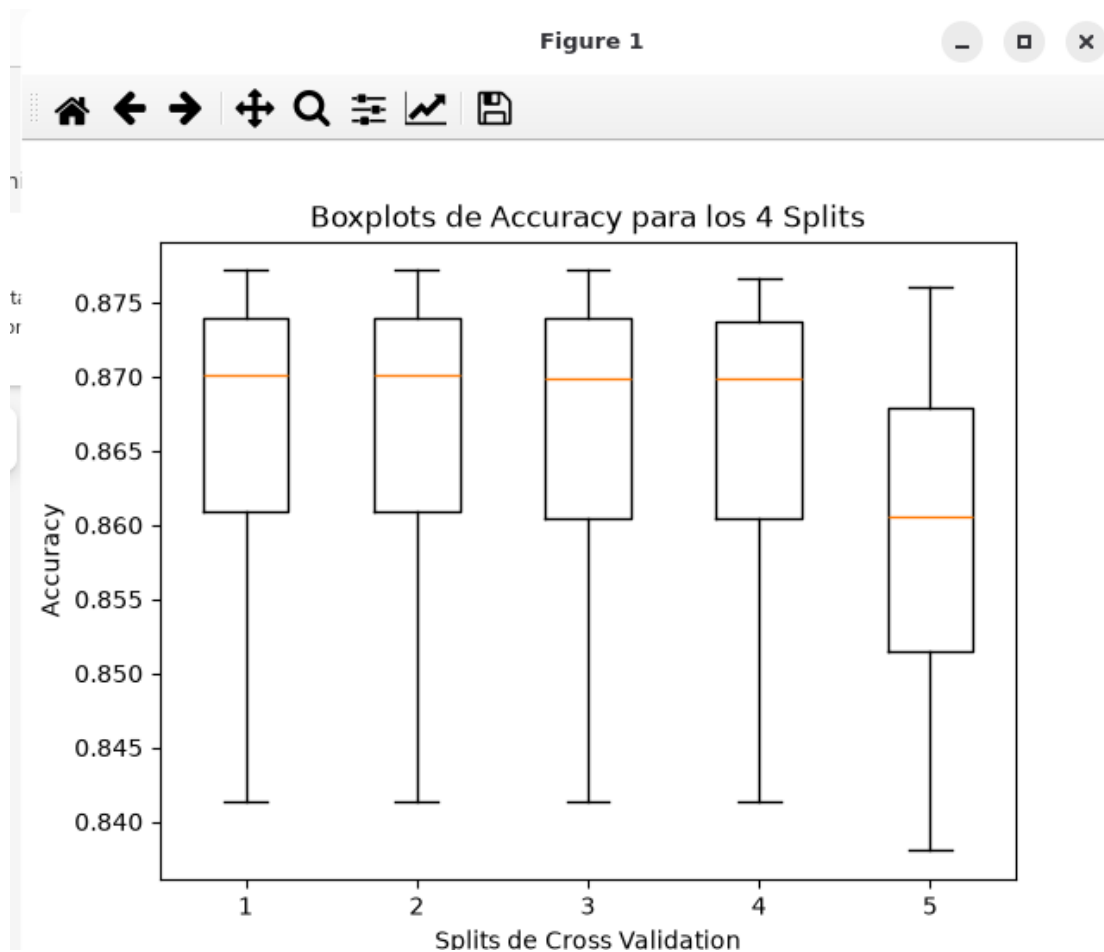
F1_score test: 0.8732394366197183

Como podemos estar viendo en estas métricas, al accuracy del train es similar al del test, por lo tanto, no se considera que esté sobreentrenado. Por otro lado, podemos ver que nos da un mejor resultado estas métricas que con el otro árbol de decisión.

De todos modos, vemos con el grid search cuál sería el mejor árbol de random forest, para ello usamos los siguientes parámetros (además de poner que me haga el refit por accuracy):

```
params_rf = {  
    'max_depth': [2, 3],  
    'min_samples_split': [4,5,10, 20],  
    'criterion': ["gini", "entropy"]  
}
```

Con estos parámetros nos salen los siguientes posibles modelos (método de validación cruzada):



Vemos que los modelos que nos salen son idénticos menos el ultimo, por lo tanto, cualquiera nos valdría. Cogemos el primero por ejemplo para calcular las métricas:

Se tiene un accuracy para train de: 0.8793478260869565

Se tiene un accuracy para test de: 0.8777173913043478

Precisión train: 0.9021864211737629

Precisión test: 0.9022118742724098

Recall/sensibilidad train: 0.8512486427795874

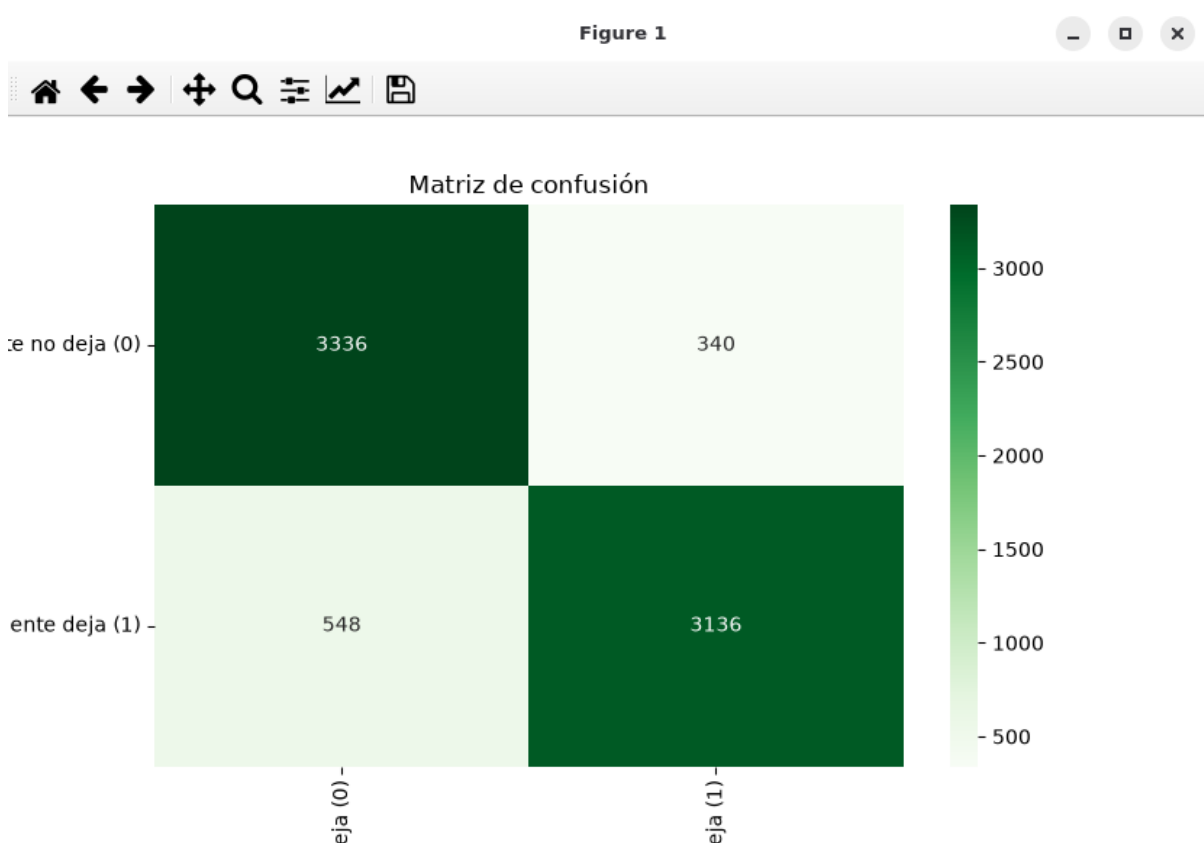
Recall/sensibilidad test: 0.8460698689956332

F1_score train: 0.8759776536312849

F1_score test: 0.8732394366197183

Como podemos ver, las métricas son idénticas al modelo visto antes, por lo tanto, hemos acertado a la primera con el mejor modelo con los parámetros indicados.

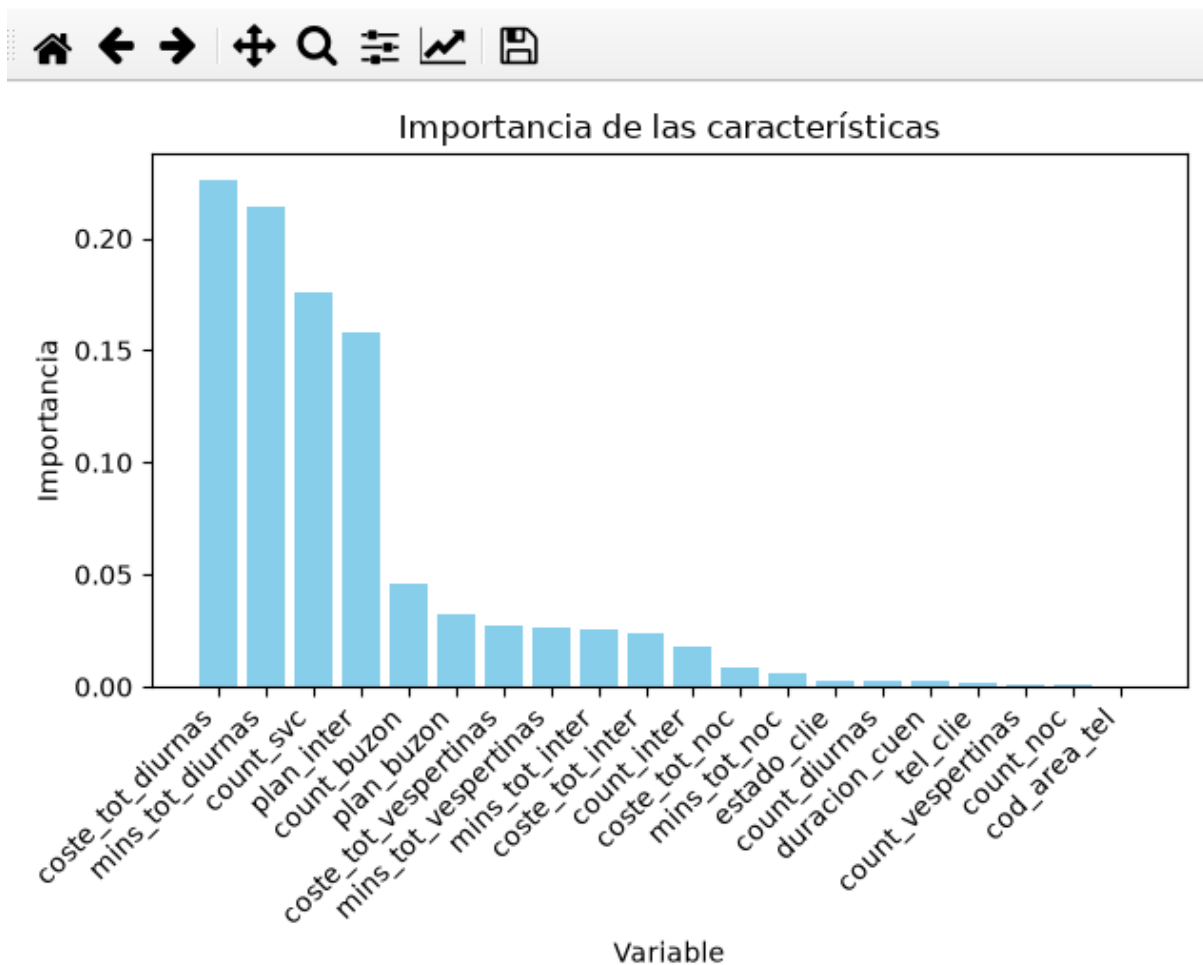
Por lo tanto, sacamos matriz de confusión:



Vemos que mejora respecto al árbol de decisión.

Como se pide también en el enunciado, vamos a ver la importancia de cada una de las variables para el modelo:

Figure 1



Como podemos observar, la variable más importante sería el coste total de las llamadas diurnas mientras que el código de área telefónica ni se utiliza.

9. XGBOOST

Creemos el árbol de clasificación usando xgboost como herramienta y lo entrenamos.

Para los parámetros de esta primera prueba usamos como booster el gbtrees y una máxima de profundidad de 3 para que no se sobreentrene:

```
arbol_xgb = XGBClassifier(booster = 'gbtree', random_state=seed, max_depth = 3, tree_method = 'hist')
arbol_xgb.fit(X_train, y_train)
```

Se tiene un accuracy para train de: 0.9623641304347826

Se tiene un accuracy para test de: 0.95

Precisión train: 0.9818953323903818

Precisión test: 0.9629213483146067

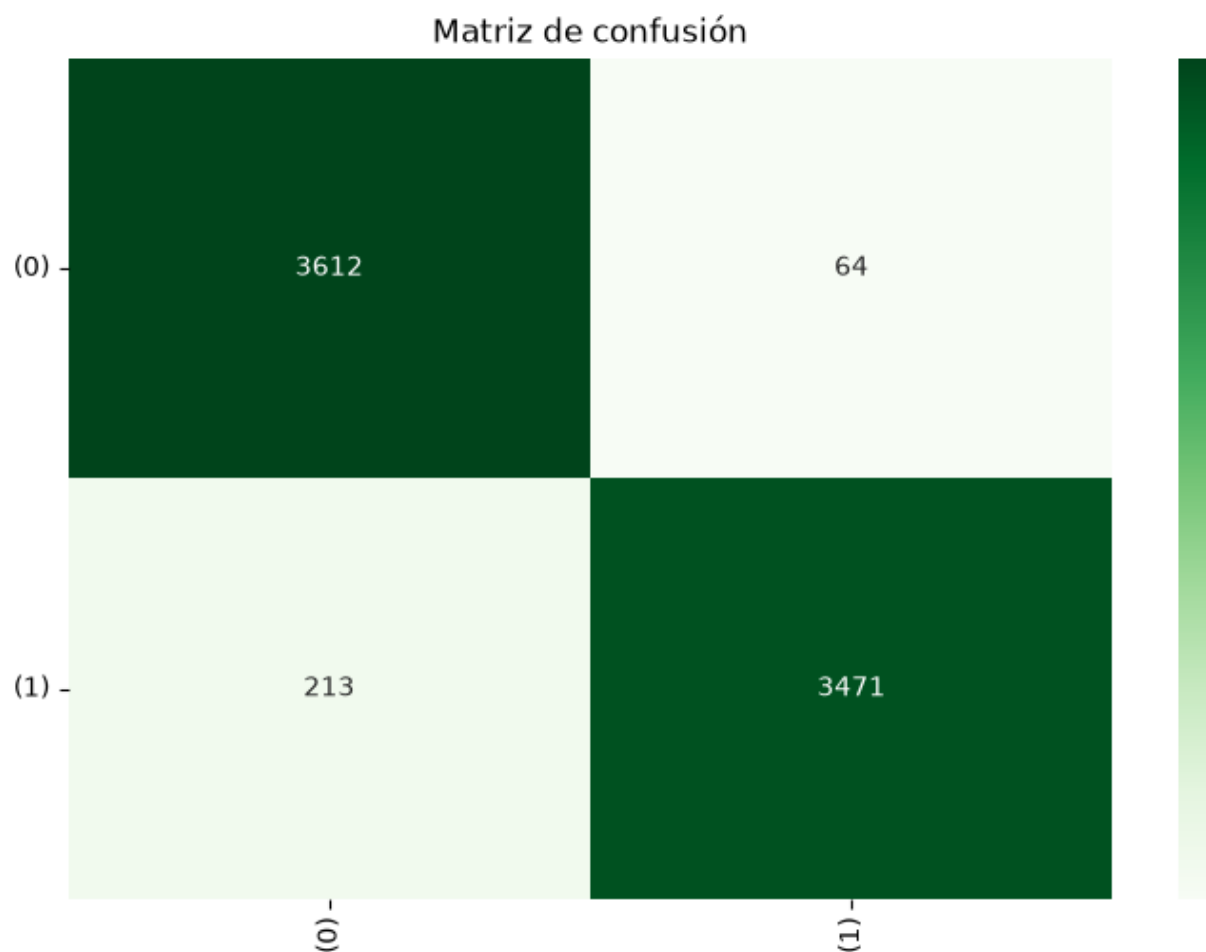
Recall/sensibilidad train: 0.9421824104234527

Recall/sensibilidad test: 0.9355895196506551

F1_score train: 0.961629034492312

F1_score test: 0.9490586932447398

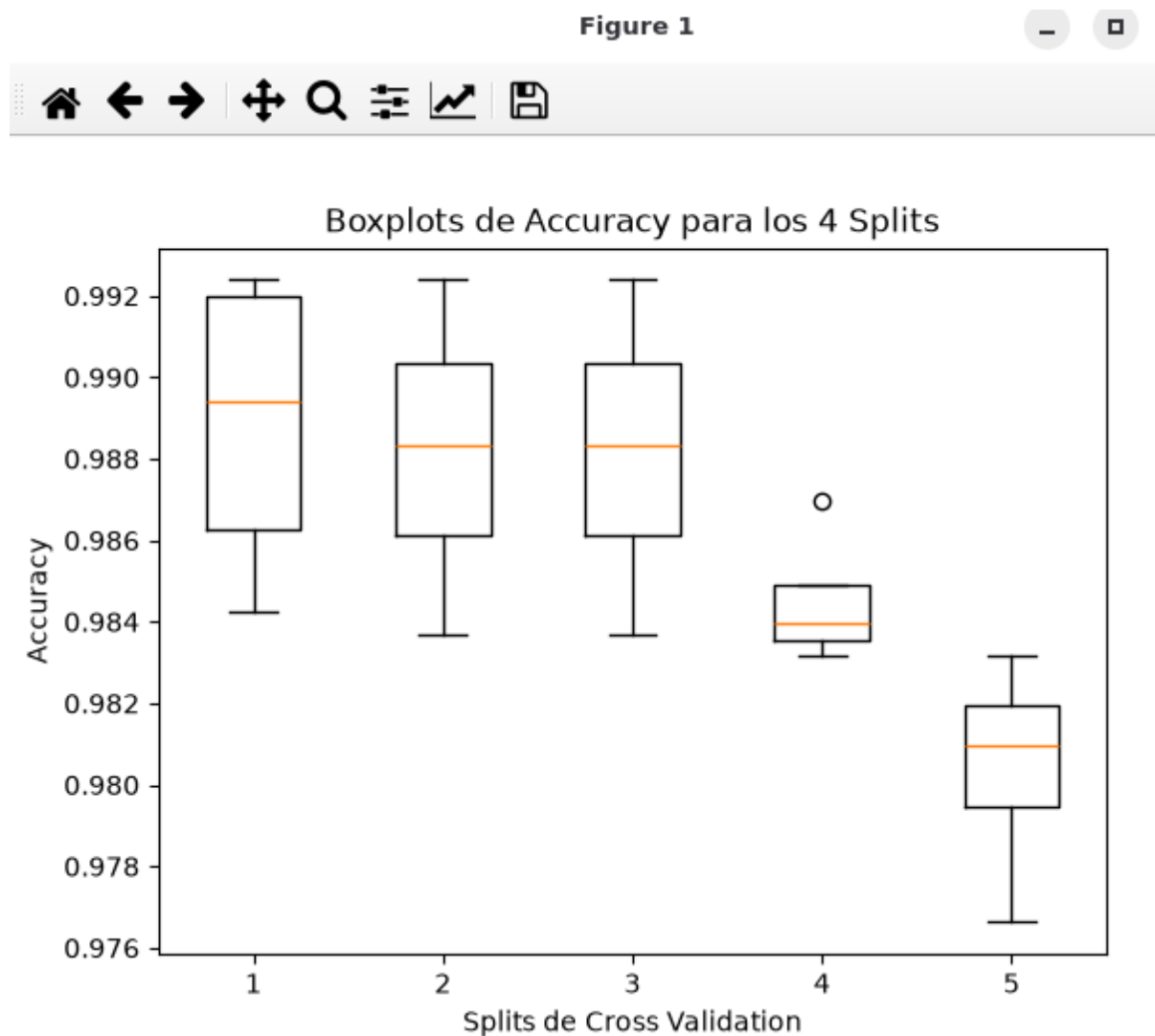
Como podemos observar con este modelo obtenemos unas mejores métricas que con el de random forest. Vemos la matriz de confusión con este modelo:



Podemos observar que clasifica mucho mejor. Pero como no sabemos si es el mejor modelo, aplicamos el gridsearch con los siguientes parámetros:

```
params_xgb = {  
    'n_estimators': [100,200,300],  
    'eta': [0.1,0.4,0.7],  
    'gamma': [0.1,0.5,1],  
    'max_depth': [2,3]  
}
```

Nos salen los siguientes modelos como los mejores:



Como podemos ver, tenemos distintas posibilidades. Esta elección iría según se quiera el modelo. Ya que, centrándonos en el primero y en el cuarto (ya que los otros son algo peores) vemos lo siguiente:

- El primero es el que mejor accuracy tiene, pero tiene más variabilidad.
- Por otro lado, el cuarto a pesar de tener un accuracy un poco peor, es más exacto y tiene menos variabilidad.

Nos quedamos con el primero ya que buscamos el accuracy. Las métricas son:

Se tiene un accuracy para train de: 0.9998641304347826

Se tiene un accuracy para test de: 0.991304347826087

Precisión train: 1.0

Precisión test: 0.9828326180257511

Recall/sensibilidad train: 0.999728555917481

Asignatura: Machine learning

Recall/sensibilidad test: 1.0

F1_score train: 0.9998642595357676

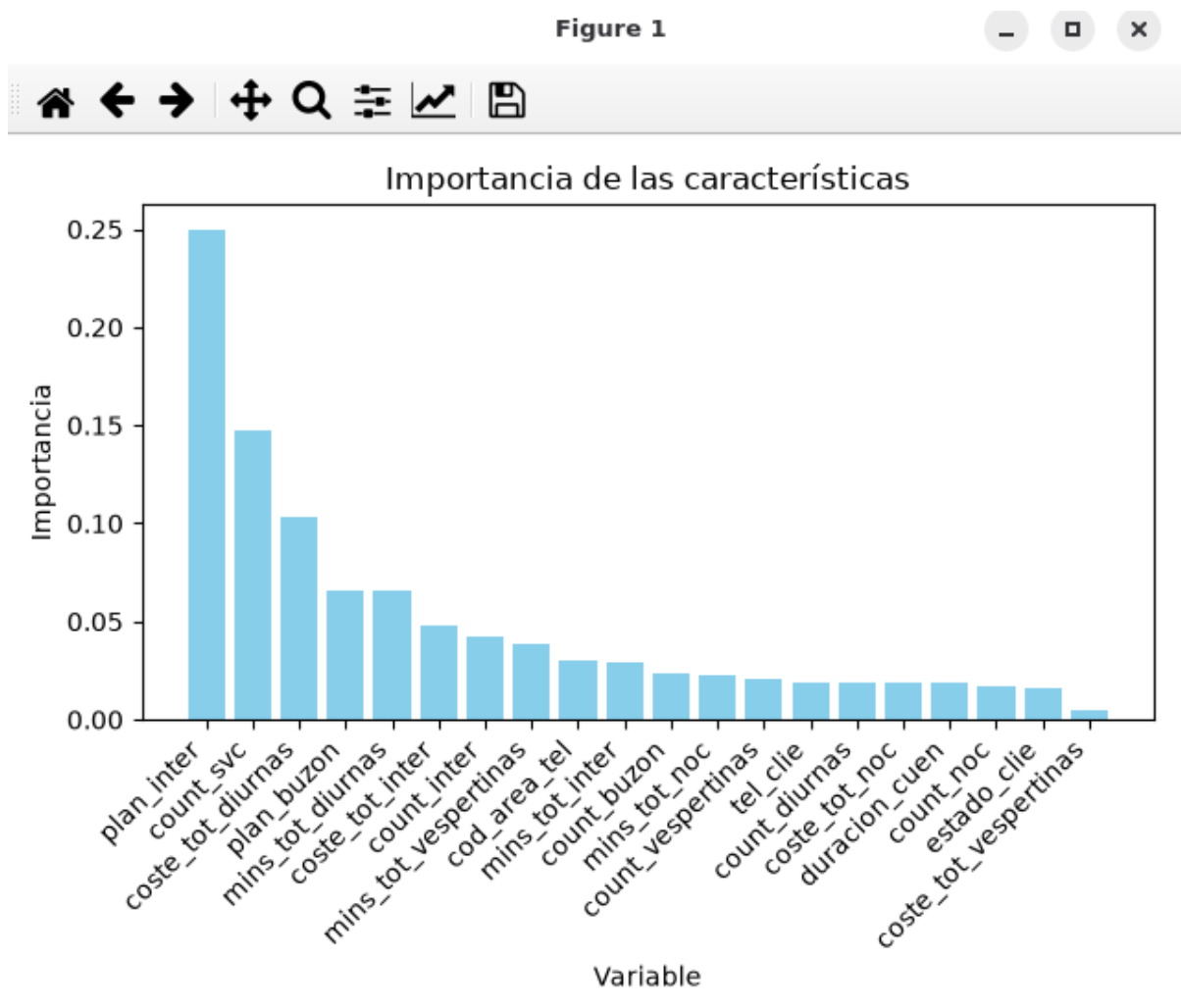
F1_score test: 0.9913419913419913

Como podemos ver, nos salen unas métricas casi perfectas. Vamos a ver la matriz de confusión ya que igual lo hemos sobreentrenado:

La matriz de confusión es la siguiente:



Como vemos con esta matriz de confusión y en las métricas se puede ver que nos han salido demasiada perfecta la clasificación, por lo tanto se podría decir que está sobreentrenado el modelo. Habría que volver a reajustarlo cambiando los aprámetros para intentar un modelo que se ajuste mejor y no esté sobreentrenado (tal vez bajando los números de estimadores). Pero vamos a seguir terminando este modelo.



Como se puede ver en esta gráfica de barras cuales son las variables más importantes. La más importante para este modelo es si tiene contratado el plan internacional, mientras que el que menos es el coste total de las llamadas vespertinas.

10. Apartado 4: Conclusión final

Según hemos estado viendo con todos los árboles que se han creado, se puede dar con las siguientes conclusiones:

- El árbol de decisión creado manualmente sin hacer uso de random forest o xgboost es más fácil poder hacer overfit mientras que en los otros dos al ir en paralelo y secuencialmente tiene menor riesgo para ello
- EL modelo de random forest tiene una forma de interpretación más alta que el xgboost pero no tanta como el árbol de decisión primero ya que podíamos seguir al 100% como estaba tomando las decisiones
- En temas de rendimiento, han ido mejor tanto el xgboost como el random forest y no tanto el árbol de decisión, pero en velocidades de entrenamiento el más rápido es el árbol de decisión mientras que los otros dos han sido parecidos.

¿Cuál sería el mejor modelo? Pues esto depende mucho de la necesidad del modelo. Si necesitásemos un modelo donde necesitamos saber que ha interpretado el modelo para tomar dicha clasificación, necesitaríamos el árbol de decisión mientras que, si necesitamos un árbol que sea preciso y que no tenga overfitting, sería más recomendable coger el random forest o el xgboost.

Al final siempre depende mucho de la necesidad del modelo.